

CASA0019: Sensor Data Visualisation 25/26

Guage and Grab Final Group Report

Group Members/Authors: Ethan Taylor; Madina Diallo; Yussr Osman Kamil Bashir

Link to the GitHub repository: <https://github.com/ethan-se/ucl-east-food-queue-gauge>

Rationale

Research Background & Project

University campus users regularly face uncertainty when choosing where to eat, especially during peak hours.

At UCL East, three key dining locations: Pool Street Café, Marshgate Café, and Marshgate Refectory, offer multiple menu options, daily specials, and a wide range of prices. However, there is currently no quick, data-driven way for students and staff to compare these choices in real time. This often leads to long queues, inefficient decision-making, and dissatisfaction & Grab was developed in response to this gap.

Our project investigates how a physical analogue device, paired with a digital-twin interface, can support users in making faster and smarter meal decisions. The concept allows users to immediately view:

- The menu options for each location (Mon–Fri)
- Daily specials and prices to help compare cost and value
- An estimated live queue-waiting time, generated through dummy / simulated data feeds via MQTT

By presenting this information through both a physical gauge (at-a-glance decision aid) and an augmented digital dashboard (expanded detail), the project aimed to reduce uncertainty and improve the everyday experience of choosing where to eat on campus.

Why We Chose This Problem?

We focused on this topic because it represents an everyday challenge that affects large numbers of people at UCL East. Our decision to help users make informed choices is grounded in three key observations:

- Time and convenience are critical factors in campus decision-making.
- Data is available but not accessible, and users cannot currently visualise queue time or menu differences instantly.
- A single, centralised display could reduce frustration, spread footfall more evenly, and improve overall campus flow.

In short, our research is framed around the question:

How can we present food-service data in a fast, simple, and meaningful way that empowers users to make informed decisions?

Data

Data Feeds Chosen

We selected three core data feeds for our prototype:

These data feeds are generated as structured dummy datasets, published through MQTT to simulate a live operational system.

Group Aim

As a group, our aim was to:

- Build a physical gauge that visualises queue time + menu status
- Create a digital twin's dashboard with expanded details and live data response
- Enable users to make a choice within 5 seconds of looking at the device
- Demonstrate how analogue and digital systems can work together for campus decision support

How We Created Our Dataset & Live MQTT System

(From market research → dataset design → live MQTT → device integration)

1. Research & Data Collection at the 3 Locations

We began by visiting each of our chosen UCL East food locations:

- Pool Street Café
- Marshgate Café
- Marshgate Canteen

Questions Asked to Staff:

Category	Example Questions
Busy Period	What days and time are the busiest?
Service Speed	<i>"How long does it normally take to serve one person on average?"</i>
Menu & Daily Specials	<i>"Do you have weekly rotations or a fixed special per day?"</i>
Bottlenecks	<i>"What slows service? Preparation? Payment? Equipment?"</i>
Queue Behaviour	<i>"At busy times, how many people are usually waiting and how do staff cope?"</i>

These answers formed the **foundation** of our dataset.

Daily Specials (Drafted from menus & staff responses)

2. Building the Dummy Dataset (Structured from Research)

We then drafted a dataset that **simulated real conditions** but remained controlled and predictable.

DAILY SPECIALS — POOL STREET CAFÉ						
Day	Daily Special	Description	Special Offer Price	Calories	Allergen	Estimated Serving Time
Monday	Pastry + Cappuccino Deal	Breakfast saver combo	£3.80	380-450 kcal	Gluten, Milk, Eggs	3 min
Tuesday	Iced Latte Day	Any iced latte	£2.20	120 kcal	Milk	2 min
Wednesday	Vegan Wrap Wednesday	Vegan wrap + fruit pot	£4.80	500 kcal	Sesame	4 min
Thursday	Cake Slice & Tea	Any cake + tea	£4.00	450 kcal	Gluten, Milk, Eggs	3 min
Friday	Student Treat Friday	Hot chocolate + brownie	£4.50	600 kcal	Milk, Gluten	3 min
DAILY SPECIALS — MARSHGATE CAFÉ						
Day	Daily Special	Description	Special Offer Price	Calories	Allergen	Estimated Serving Time
Monday	Mozzarella Panini Special	Panini + soft drink	£5.00	730 kcal	Milk, Gluten	5 min
Tuesday	Smoothie Boost Day	Any fresh smoothie	£2.50	160 kcal	None	2 min
Wednesday	Wrap & Water Wednesday	Chicken or Vegan wrap + water	£4.80	550-620 kcal	Gluten, Milk, Sesame	4 min
Thursday	Croissant & Latte Treat	Fresh pastry + latte	£4.20	400-500 kcal	Gluten, Milk, Eggs	3 min
Friday	Bagel Friday	Smoked salmon bagel	£4.90	480 kcal	Fish, Milk, Gluten	3 min
DAILY SPECIALS — MARSHGATE CANTEN / REFECTORY						
Day	Daily Special	Description	Special Offer Price	Calories	Allergen	Estimated Serving Time
Monday	Plant Power Bowl	Hot vegan grain bowl	£5.20	650 kcal	None	6 min
Tuesday	Chicken Katsu Day	Chicken katsu curry + rice	£6.00	1050 kcal	Gluten, Milk, Eggs	7 min
Wednesday	Mac + Chicken Bowl	Mac + Cheese + chicken	£5.80	1115 kcal	Milk, Gluten	6 min
Thursday	Veg Lasagne Deal	Lasagne + salad	£5.80	950 kcal	Milk, Gluten	7 min
Friday	Fish & Chips	Classic fish & chips	£6.50	1130 kcal	Fish, Gluten	8 min

Data Feed Type	Purpose	Output Example
Menu Data (Mon–Fri)	Shows available items without searching online	“Chicken Wrap – £4.50”
Daily Special & Prices	Encourages discovery + better value choices	“Wednesday: Mac &Chicken Bowl – £5.80”
Live Queue Time Estimate (MQTT Dummy Data)	Helps users choose based on waiting duration	“Approx. 7 minutes wait”

Insights We Used

Location	Avg. Queue	Speed per person	Multiplier
Pool Street Café	Small queue	Fast	×1
Marshgate Café	Medium	Normal	×2
Marshgate Canteen	Larger queue	Slowest	×3

Queue Time Formula

This formula became our standard:

queue_time = number_of_people × service_factor

Final Data Structure (JSON format used later for MQTT)

```
{
  "location": "pool_street",
  "people_in_queue": 6,
  "service_factor": 1,
  "queue_time": 6,
  "daily_special": "Cake Slice & Tea",
  "price": "6.50"
}
```

3. Designing Topics for MQTT (Data Architecture)

To publish data properly, we needed a logical topic hierarchy to separate each location:

```
student/  
  CASA0019/  
    gauge&grab/  
      pool_street  
      marshgate_cafe  
      marshgate_canteen
```

Each device subscribes only to the location it needs.

4. Creating Live Data in Python (DaBo Publisher Script)

main.py

```
import time, random, json  
from mqtt_publisher import MQTTClient  
  
# MQTT connection  
client = MQTTClient("mqtt.cetools.org", 1884, "student", "ce2024")  
  
# Locations + service multipliers  
locations = {  
    "pool_street": 1,  
    "marshgate_cafe": 2,  
    "marshgate_canteen": 3  
}  
  
# Daily specials  
specials = {  
    "pool_street": ("Vegan Wrap & Fruit Pot", "4.80"),  
    "marshgate_cafe": ("Panini Soft Drink", "5.00"),  
    "marshgate_canteen": ("Fish & Chips", "6.50")  
}  
  
while True:  
    for location, factor in locations.items():  
        people = random.randint(1, 20)  
        queue_time = people * factor # formula from research  
  
        data = {  
            "location": location,  
            "people_in_queue": people,  
            "queue_time": queue_time,  
            "daily_special": specials[location][0],  
            "price": specials[location][1]  
        }
```

```
topic = f"student/CASA0019/gauge&grab/{location}"
client.publish(topic, json.dumps(data), retain=True)
print("Published → ", topic, data)
```

```
time.sleep(2)
```

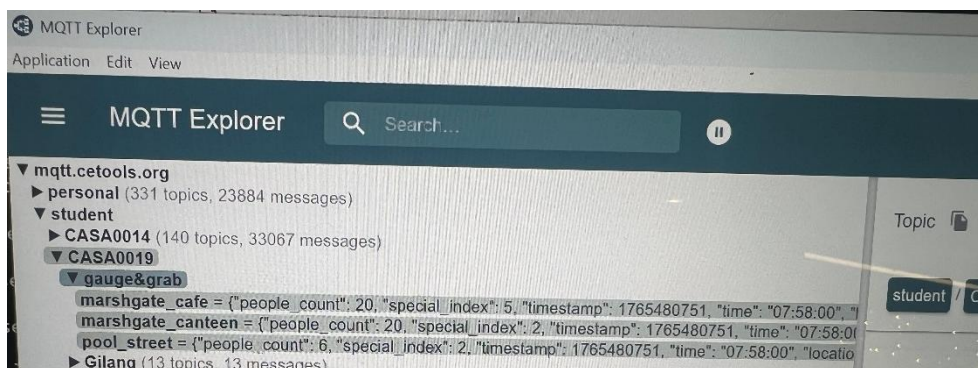
Result: Every 2 seconds, live data gets published to all three topics.

5. Data Arrives in MQTT Explorer (Verification Step)

To confirm everything works:

1. Open **MQTT Explorer**
2. Add broker:
3. Host: mqtt.cetools.org
4. Port: 1884
5. Username: student
6. Password: ce2024
7. Subscribe to:
8. student/CASA0019/gauge&grab/#

live queue data updating in real time.



6. Subscribing with the Physical Gauge (Arduino)

Arduino Pseudocode Subscribe Logic

```
client.subscribe("student/CASA0019/gauge&grab/pool_street");
```

```
if(messageArrived){
  queue_time = payload["queue_time"];
  moveGaugeNeedle(queue_time);
  lcd.print(payload["daily_special"]);
}
```

The gauge needle + LCD now update in real time.

7. Unity Digital Twin Subscription (C# Script)

mqttManager.cs

```
client.Subscribe("student/CASA0019/gauge&grab/#");
```

```

void OnMessageArrived(string topic, string message) {
    var data = JsonUtility.FromJson<GaugeData>(message);
    dashboard.Update(data.queue_time);
    graph.AddPoint(data.queue_time);
}

```

The AR dashboard + graphs update based on the same live feed.

8. Summary of the System Pipeline

Real Questions at Cafés



Draft dataset (menus, multipliers, queue ranges)



Python generator → Live values every 2 seconds



MQTT Publish → student/CASA0019/gauge&grab/<location>



Physical Gauge Reads (Arduino)



Digital Twin Reads (Unity AR)

Group Reflection

Limitations

Despite its effectiveness as a prototype, the system has several limitations:

- The dataset is simulated and does not reflect real-time operational data
- Queue sizes are randomly generated rather than sensor-derived
- Service speed is simplified into fixed multipliers
- User-specific factors (dietary needs, budget constraints) are not currently included
- Long-term data persistence and historical storage were not implemented

These limitations were accepted to prioritise system architecture, interaction design, and real-time data flow within the project scope.

Future Work

Future development could address these limitations by:

- Integrating real sensors (e.g. people counters or computer vision) for live queue detection
- Connecting to real menu management or point-of-sale systems
- Applying machine learning to predict queue times based on historical trends
- Adding personalisation features such as dietary preferences or budget filters
- Developing a mobile application to complement the physical gauge

- Providing a staff-facing dashboard for operational decision-making

These enhancements would transform Gauge & Grab from a demonstrative prototype into a fully deployable smart-campus service.

“A future improvement would allow users to submit short, optional feedback after visiting a café, enabling us to compare lived experience with system estimates and continuously improve queue-time accuracy.”

“The project was structured with a dedicated virtual environment separated from the application code to ensure reproducibility, clean dependency management, and ease of deployment. All application logic, configuration, and documentation were contained within a single project directory suitable for version control.”

References:

- Eclipse Foundation (2024) paho.mqtt.python: Eclipse Paho MQTT Python client library. Available at: <https://github.com/eclipse-paho/paho.mqtt.python>
- Eclipse Mosquitto (2025) MQTT: lightweight messaging protocol. Available at: <https://mosquitto.org/>
- Python Software Foundation (2025) Python documentation. Available at: <https://www.python.org/doc/>
- Schiffler, A. (2025) How to use the Paho MQTT client in Python with examples. Cedalo Blog. Available at: <https://cedalo.com/blog/configuring-paho-mqtt-python-client-with-examples/>
- UCL Centre for Advanced Spatial Analysis (2025) DaBo: Data in a Box. Available at: <https://github.com/ucl-casa-ce/dabo>